

This file behaves like any other file on your computer. You can open it, write new text in it, copy from it, paste to it, and so forth.

NOTE

Python can only write strings to a text file. If you want to store numerical data in a text file, you'll have to convert the data to string format first using the `str()` function.

Writing Multiple Lines

The `write_text()` method does a few things behind the scenes. If the file that path points to doesn't exist, it creates that file. Also, after writing the string to the file, it makes sure the file is closed properly. Files that aren't closed properly can lead to missing or corrupted data.

To write more than one line to a file, you need to build a string containing the entire contents of the file, and then call `write_text()` with that string. Let's write several lines to the `programming.txt` file:

```
from pathlib import Path

contents = "I love programming.\n"
contents += "I love creating new games.\n"
contents += "I also love working with data.\n"

path = Path('programming.txt')
path.write_text(contents)
```

We define a variable called `contents` that will hold the entire contents of the file. On the next line, we use the `+=` operator to add to this string. You can do this as many times as you need, to build strings of any length. In this case we include newline characters at the end of each line, to make sure each statement appears on its own line.

If you run this and then open `programming.txt`, you'll see each of these lines in the text file:

```
I love programming.
I love creating new games.
I also love working with data.
```

You can also use spaces, tab characters, and blank lines to format your output, just as you've been doing with terminal-based output. There's no limit to the length of your strings, and this is how many computer-generated documents are created.

NOTE

Be careful when calling `write_text()` on a path object. If the file already exists, `write_text()` will erase the current contents of the file and write new contents to the file. Later in this chapter, you'll learn to check whether a file exists using `pathlib`.